

Insumo para IA de presentaciones -

main.js

Documento pensado para que otra IA pueda armar una presentación de producto/arquitectura sin leer todo el repo. Este resumen está centrado en el archivo `main.js`, que contiene la lógica principal de frontend de Pandi.

10.218

líneas aprox. en `main.js`

100

funciones declaradas

22

funciones `setup*`

13

funciones `load*`

11

funciones `render*`

8

funciones `save*`

1) Qué resuelve este archivo

- Autenticación y sesión con Supabase (login, registro, signOut, permisos por rol).
- Navegación SPA y carga de vistas: Inicio, Órdenes, Cajas, Clientes, Intermediarios, Tipos de operación, Cuenta corriente, Seguridad.
- Motor operativo de órdenes/transacciones y sincronización contable: **CC + Caja**.
- Aplicación de reglas de negocio de CC desde tabla `cc_modelo_reglas` (fuente de verdad).
- Modales, UX (drag, backdrop real click), ayudas, toasts, confirmaciones y exportes Excel.

2) Dependencias y contrato técnico

`main.js` depende de:

Supabase JS client (`window.supabase`)

Config runtime: SUPABASE_URL / SUPABASE_ANON_KEY

XLSX vía CDN (`window.XLSX`)

`utils.js` (formatos + iconografía)

Si falta configuración, corta la ejecución con mensaje de error visible en pantalla.

3) Módulos funcionales (visión de producto)

Seguridad y permisos

- Alta/perfil de usuario (`ensureProfile`).
- Permisos por menú y por operación (`loadSeguridad`, `refreshPermisosYVista`).
- Sesión por inactividad configurable (Admin).

Inicio y pendientes

- Cards de caja + paneles de pendientes.
- Modales de órdenes/transacciones pendientes con filtros.
- Recarga suave cada 30 segundos (si no hay modal/expandido).

Cajas

- Cálculo de saldos desde movimientos.
- ABM de movimientos y tipos de movimiento.
- Separación por caja (efectivo/banco/cheque) y moneda.

Cuenta corriente (core)

- Resumen por cliente/intermediario y detalle filtrable.
- Export Excel de resumen/detalle.
- Reglas dinámicas de signo/exposición/saldo desde `cc_modelo_reglas`.

Órdenes e instrumentación

- Wizard de orden y transacciones.
- Tipos de operación con/sin intermediario, incluyendo CHEQUE-ARS.
- Sincronización contable al cambiar estado de transacciones.

UX transversal

- Modales arrastrables y cierre seguro por backdrop real.
- Ayudas con modal (`setupHelpPopovers`).
- Mensajería interna (`showToast`, `showConfirm`).

4) Motor CC por tabla (mensaje clave para presentación)

El diseño buscado es: **“la tabla de reglas es la única fuente de verdad”**. El frontend no decide reglas financieras hardcodeadas, sino que consulta y aplica filas de `cc_modelo_reglas`.

- `getReglasCcModelo`: trae reglas por tipo de operación y uso de intermediario.
- `lookupRegla`: selecciona regla exacta por pagador/cobrador/tipo/comisión/estado/contrapartida.

- `resolverMonedaYMontoCcClienteMotor` y `resolverMonedaYMontoCcIntermediarioMotor`: resuelven moneda/monto de exposición.
- `estadoEfectivoComision`: interpreta condiciones de comisión sin transacción propia.
- `sincronizarCcYCajaDesdeOrden`: aplica reglas y reconstruye movimientos de CC y caja.

Punto narrativo fuerte para slides: Pandi desacopla "motor de cálculo" del "código UI". Cambiar reglas de negocio implica cambiar datos/reglas SQL y no reescribir todo el frontend.

5) Flujos end-to-end importantes

Flujo A - Login y arranque

1. `client.auth.getSession()` verifica sesión.
2. Si no hay sesión: `showLogin()` + `setupLoginAndRegister()`.
3. Si hay sesión: `onSessionReady(session)`.
4. Se cargan permisos, timeout, setup de modales/vistas, y vista inicial permitida.

Flujo B - Cambio de vista

1. `showView(vistaId, pageTitle)` controla visibilidad SPA.
2. Dispara `load*` correspondiente.
3. Cuenta corriente usa loading previo para evitar "flash" de saldos viejos.

Flujo C - Impacto contable por transacciones

1. Usuario cambia estado de transacciones en orden/instrumentación.
2. Se sincroniza vía `sincronizarCcYCajaDesdeOrden`.
3. Se aplican reglas de `cc_modelo_reglas`.
4. Se reflejan resumen y detalle de CC + saldos de caja.

6) Funciones clave para mostrar en slides técnicas

`setupLoginAndRegister`

`showView`

`loadSeguridad`

`loadInicio`

`loadCajas`

`loadCuentaCorriente`

`getReglasCcModelo`

`lookupRegla`

`estadoEfectivoComision`

`resolverMonedaYMontoCcClienteMotor`

`resolverMonedaYMontoCcIntermediarioMotor`

sincronizarCcYCajaParaTodasLasOrdenesConInstrumentacion

exportarCcResumenExcel

setupModalesDraggable

setupHelpPopovers

7) Puntos de negocio que NO hay que omitir en la presentación

- **CC y caja deben cerrar siempre** con la realidad operativa.
- Cliente e intermediario se modelan en paralelo (coherencia global).
- Reglas de CC parametrizadas por tabla (motor data-driven).
- E2E de combinaciones valida estados P/E y evita regresiones financieras.
- Exportación Excel numérica para análisis contable real.

8) Estructura sugerida de slides (lista para IA)

1. Problema: complejidad de CC/caja y riesgo de reglas hardcodeadas.
2. Solución: motor de reglas `cc_modelo_reglas` + sincronización automática.
3. Arquitectura funcional del frontend (`main.js` como orquestador).
4. Flujo de negocio de una orden (de instrumentación a impacto contable).
5. Casos de prueba E2E por combinaciones de estados.
6. Beneficios: mantenibilidad, auditabilidad, velocidad de cambio.

9) Referencias para ampliar

- Código fuente completo: `/main.js`
- Motor de reglas y docs: `docs/CC_MODELO_ENGINE_TABLA.md`
- Reglas SQL: `sql/cc_modelo_reglas_tabla.sql`
- E2E combinaciones: `tests/e2e/cc-combinaciones.spec.js` y `tests/e2e/cc-tipos-activos-combinaciones.spec.js`

Archivo generado para insumo de presentación. Fecha de referencia: marzo 2026.